SAPIENS
Decision

Decision Execution (DE) Installation and Configuration Guide

Document Version : 7.6
Software Version: 7.6

Partnering for Success

SAPIENS

Decision

July 2025

Partnering for Success

Notice

Trademarks

Statement of Conditions

Caution

Any changes or modifications of the software not expressly approved in writing by the manufacturer could void the user's authority to use the application and its warranty.

Document Revision History

Revision	Date	Author	Description	Approved by
7.5.2	May 2025	Decision Documentation	Describes the DE 7.5.2 Installation and Configuration information	Decision Product Management
7.6	July 2025	Decision Documentation	Describes the DE 7.6 Installation and Configuration information	Decision Product Management

Document Authorization

This document is digitally signed and authorized by the Sapiens Decision documentation team. The steps to authenticate the digital certificate are documented in the Index file present in the documentation package. In case of any queries or feedback on the document, send an e-mail to Decision_documentation@sapiens.com.

Table Of Contents

1. About this Document

This guide provides installation and configuration instructions for Decision Execution (DE).

DE can be used on various databases such as PostgreSQL, Oracle, and MS-SQL Server.

For more details on the database setup, navigate to the specific sections for the implementation of the database on DE.

The instructions in this guide are mainly applicable for the Linux operating system and PostgreSQL database server platform, although additional information is provided, where applicable, for installing Windows OS and/or Oracle DB server / MS-SQL DB server systems.

In addition to the installation procedures, instructions for upgrades are provided to detail the changes required in existing installations.

Prerequisites and Notes

- This guide assumes technical knowledge in the relevant operating system, RDBMS, Tomcat setup, and security requirements.
- Installation setup assumes access to RDBMS is provided - Oracle, MS-SQL Server, and PostgreSQL, with the required DBA rights.
- All firewall definitions and access rights to the different services are assumed ready.

Related Documents

For information about the latest DE release, refer to the latest *Decision Execution (DE) Release Notes*.

For information about the latest Decision Manager (DM) release, refer to the latest *Decision Manager (DM) Release Notes*.

2. System Requirements

The following are the optimum requirements to run the DE server.

Note: Java JDK is not part of the application package. You must manually install the JDK recommended version mentioned in the table.

Software Requirements	
Java Application Server	Certified with Tomcat 10.1.42
Database	**DE schema on either:** Oracle - 19 Microsoft - SQL 2022 PostgreSQL DB – 15
Operating System (OS) with Java support	Certified with Red Hat Enterprise Linux 8 and 9.2, AWS Linux 2
Docker	19.x, 20.x
Kubernetes	Client Version: v1.28.4 Customize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
Helm	3.14.4
Java JDK	Oracle – 17.0.14 Amazon-Corretto - 17.0.14.7.1 Red Hat OpenJDK – 17.0.14

Hardware Requirements	
Hardware	**Recommended Storage**
CPU	2 CPU with 2.5 GHz or above
RAM	8 GB
Java heap size	4 GB - recommended
Hard disk	50 GB

3. Version Compatibility

For version compatibility information for the Decision Execution (DE) application, refer to the **Version Compatibility** section in the *Decision Execution (DE) Release Notes*.

4. Installation

The following section describes the detailed DE Installation procedure. This document provides the instructions for a base Decision Execution Installation. Refer to the Advanced Configuration Topics section for advanced installation options such as Clustering, Secured Properties, Custom Context Path, and HTTPS Configuration.

Note: The DM internal DE is installed unsecured for testing on the same Tomcat, while the standalone DE is installed on a separate Tomcat.

Important changes to note in this DM and DE Release Installation and Configuration Guide are:

1. The DE installation procedure must be performed for both “DE Standalone” and “DM Internal DE”.
2. The DE PostgreSQL schema has been modified. As a result, the dialect has been updated, and both DDL and DML SQL scripts must be executed. Refer to the Upgrade Procedure section.
3. As per the DM Internal DE request, update the application.properties file by adding the following entry:

- decision.de.db.repository.monitor.disabled=true

4. No updates are needed for DE Standalone.

Required DB privileges – DM, DT, and DE Schemas

The following privileges are required:

- Create/Update/Delete Table
- Session
- View
- Sequence

Installation Stages

The installation process involves the following stages:

1. Download the DE release package from the Sapiens Azure storage location as provided in the release notification email or request the location from the Decision Support team. The release package includes schema DDL, configuration files, and war file for the DE server.
2. Create the DE schema in your RDBMS.
3. Deploy war and conf files.
4. Setup conf files.

The above is provided as per Tomcat installation and files in the configuration directory (conf). To modify, you must have the following:

- Tomcat access to RDBMS is required.
- Tomcat setup for http or https, and access to service is required

5. DE Deployment & Installation

5.1 Overview

Decision supports multiple deployment styles, each with a specific package provided. Although these packages are logically identical, they differ in layout, installation, configuration, platform dependencies, and startup mechanisms/commands.

5.2 Containerized Image Deployment

Containerized image deployment is a method where application components are packaged as Docker images, providing flexibility, scalability, and ease of management across various containerized platforms.

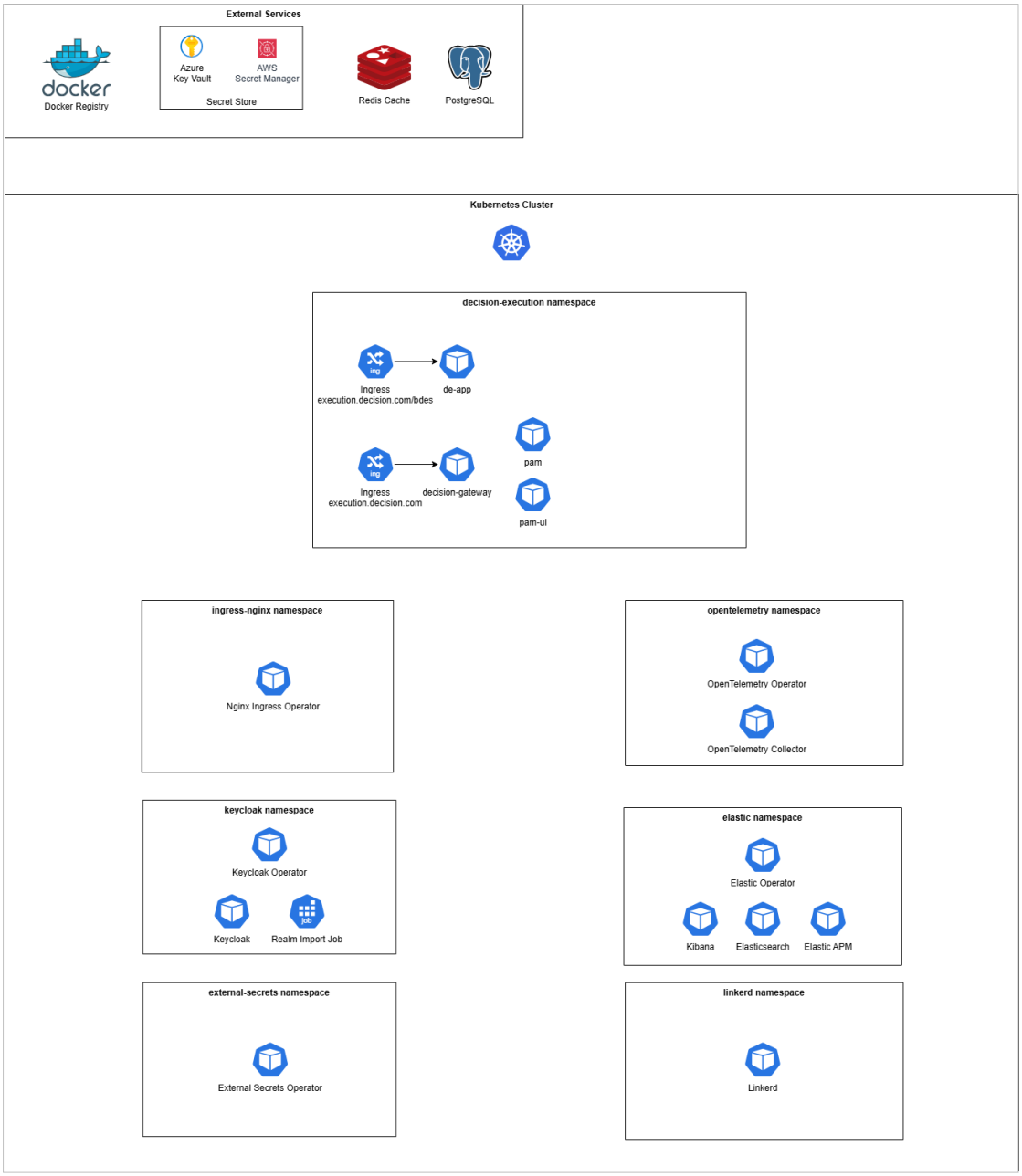
5.2.1 Overview

The containerized deployment is an advanced form of standalone deployment, where Sapiens provides the components as JAR files packaged within Docker images. These images can be deployed in various containerized environments, including Kubernetes, OpenShift, AKS, and EKS.

For detailed deployment instructions, refer to the Helm chart's README.md file, as explained in the Accessing the README.md File for Deployment Instructions section.

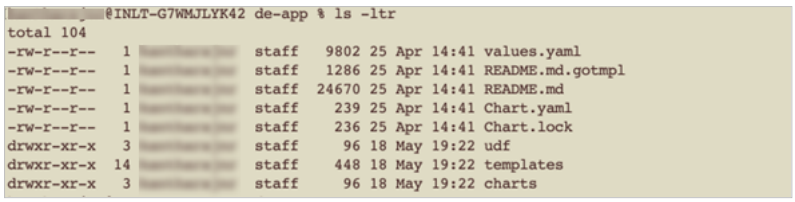
Advantages of Containerized Deployment:

- **Pre-Packaged Application JARs**: Application JARs are bundled within Docker images, simplifying deployment.
- **Isolated Failure Points**: A failure in one application component does not affect others, except where dependencies exist.
- **Flexibility and Scalability**: Components can be independently scaled and managed.
- **Fast Startup**: Each application component can be rapidly started in isolation.
- **Resource Optimization**: Independent memory and resource allocation for each component, tailored to its requirements.
- **Enhanced Monitoring**: Each application runs as an independent Java process, enabling granular monitoring.
- **Fine-Grained Scaling**: Individual components can be scaled as needed.



5.2.1.1 Accessing the README.md File for Deployment Instructions

- Locate the downloaded package file.
- Extract the package using any archive tool (e.g., WinRAR, 7-Zip).
- Navigate to the extracted folder and open the **de-app** directory.
- The **README.md** file should be located in the root of this extracted folder.



Helm chart's README.md file

5.2.2 Package Content

This section lists the essential components and resources included in the containerized deployment package, ensuring all necessary files and images are available.

- **Docker Images** and files for each of the components of the application.
- **Helm Charts**: Configuration files to manage the deployment of these Docker images in Kubernetes environments.

5.2.3 Recommended Use Cases

This section provides scenarios where containerized deployment is most beneficial, helping users decide when to opt for this model.

Containerized deployment is recommended in the following scenarios:

- Deployment to a containerized environment (e.g., Kubernetes, OpenShift, AKS, EKS) is required.
- Preference for pre-packaged Docker images for simplified deployment.
- Requirement for per-application scalability, resource allocation, monitoring, and isolation.
- Environments requiring fast recovery and startup of each application component.
- High-availability production environments with a large number of users and/or high request rates.

5.2.4 Prerequisites

This section outlines the system and platform requirements necessary for a successful containerized deployment.

Ensure that the following platform dependencies are met before initiating containerized deployment:

- **Compatible Containerized Environment**: Kubernetes, OpenShift, AKS, or EKS.
- **DE:**
    - Docker Image
    - Database schema with all read/write permissions on any of the schema objects (tables, sequences, etc.). Refer to the Database Implementation section for database-specific instructions.
- **Decision Gateway**:
    - Docker Image
    - Redis Database.
- **Configuration Service (Optional)**:

- Docker Image for RabbitMQ.
- Rabbit MQ:
  - **Docker Image for RabbitMQ**: This is a pre-built, standardized package that allows you to run RabbitMQ easily using Docker.
  - **bitnami/rabbitmq:4.1.0-debian-12-r0**: This indicates a specific version of the RabbitMQ Docker image, with the "management" plugin enabled. The management plugin provides a user-friendly web-based management UI for RabbitMQ.

- **OAuth IDP (e.g., Keycloak)**: Required for user authentication using OAuth 2.0.
- **Ingress NGINX Controller**: For external access management.
- **APM Tool (Optional)**: Application Performance Monitoring (e.g., Elastic ECK).
- **External Secrets Operator (Optional)**
- **OpenTelemetry Configuration (Optional)**:

  OpenTelemetry is an open-source observability framework for generating, collecting, processing, and exporting telemetry data (like traces, metrics, and logs) from applications.

  Components of OpenTelemetry Configuration:
  - OpenTelemetry Operator
  - OpenTelemetry Collector
  - AutoInstrumentation Configuration

5.2.5 Installation

For detailed installation instructions, refer to the Helm chart's README.md file, as explained in the Accessing the README.md File for Deployment Instructions section.

5.3 "All-in-One" Deployment

The all-in-one deployment model is a traditional approach where all Decision components are deployed on a single host machine, operating under one application server (such as Tomcat). This approach simplifies management but has limitations in scalability and resource optimization.

5.3.1 Overview

The following are brief descriptions of prerequisite folders included in the current DE release and mentioned throughout this document.

Folder	Description
Administration	WSDL files for setting up the SOAP endpoint that will call DE
DDL	DDL files for Postgres, Oracle, and MS-SQL
Conf	Configuration files for DE configuration
War	App WAR file to be installed on the Tomcat Application Server

Folder	Description
Release Notes	The Release Notes document

Note: When the installation is complete, a DE instance will be running once Tomcat is started.

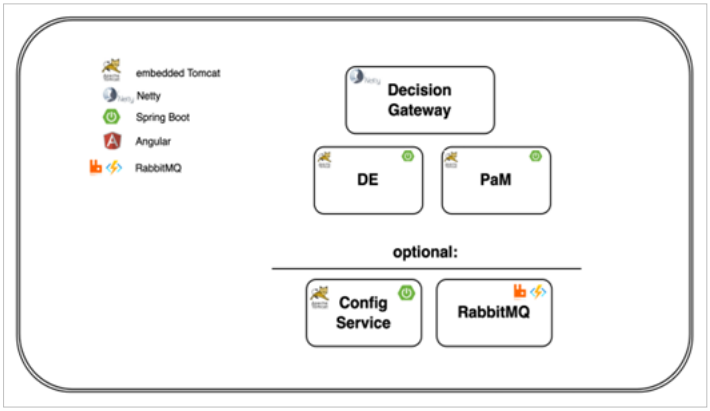
The term 'all-in-one' describes a deployment model where all Decision components are packaged and deployed together on a single host under one application server. This model is straightforward but has several advantages and disadvantages.

Advantages:

- A single configuration and maintenance point for all Decision components.
- No network delay between components since they run on the same host.

Disadvantages:

- Single point of failure – an issue with one component can impact the entire application.
- No fine-grained resource allocation for individual components.
- Larger application archive and memory footprint.
- Longer scaling time, as all components must be scaled together.



All-in-One deployment Architecture

5.3.2 Package Content

This section details the components included in the all-in-one deployment package.

- **Web Application Archives (WAR)**: Application components are packaged as WAR files, which are deployed under a single application server (e.g., Tomcat).

5.3.3 Recommended Use Cases

The all-in-one deployment method is recommended in the following scenarios:

- The customer prefers to host the Decision application on an existing web application server.
- The convenience of a single application server configuration, a centralized deployment location, and a single port is prioritized over per-application instance scalability, resource allocation, monitoring, and isolation.

5.3.4 Prerequisites

This section outlines the essential system and platform requirements for deploying Decision using the all-in-one deployment model. Ensure all the following prerequisites are met before starting the installation.

Platform Requirements:

- **Application Server:**
  - Tomcat Application Server (version 10 or above)
- **Java Development Kit (JDK):**
  - Installed JDK distribution of Java 17
  - The JDK installation path should be set in the JAVA_HOME environment variable, either by the operating system or directly within the Tomcat application server.
- **Disk Space:**
  - Minimum of 2GB available disk space (including the Tomcat server).
- **Memory (RAM):**
  - For basic usage (application startup and one concurrent user): At least 3GB of RAM
  - For optimal performance in most production environments: At least 8GB of RAM.

Database Requirements:

- **Database Server:**
  - PostgreSQL or Oracle (refer to each application installation guide for minimum supported versions and database-specific instructions).
- **Dedicated Database Schema:**
  - Each Decision application must have a dedicated schema with full read/write permissions on all schema objects (tables, sequences, etc.).
  - Multiple schemas can exist under the same database server instance, but each must be isolated per application.

Application Package (WAR Files):

The following web application archive (WAR) files are required for a complete all-in-one deployment:

- **Mandatory Components:**
  - `bdes-7.5.2.war` - Decision Execution (DE) Application
- **Optional Components (Health Monitoring):**
  - `o health-service-0.0.13.war` - Health Service Application

5.3.5 Installation

This section provides a detailed, step-by-step guide for installing the Decision components in an all-in-one deployment on a Tomcat application server. Ensure you complete all prerequisite steps before proceeding with the installation.

5.3.5.1 Rule Repository Configuration

Choose one of the following options.

The rule repository is where all deployment-related data is stored. The repository can be of two types:

- Database: Includes all deployed assets.
- File System: Includes deployed assets and additional system information.

Database Repository

Edit {DE_HOME}/conf/de-config/bdes.cust.properties by adding the following property:

- spring.profiles.active =db-repository

For database configurations, refer to the Database Implementation section.

File System Repository

Under an accessible directory, create a directory filesystem, and under it, create sub-directories rules-repo, temp and export.

For this guide, we will assume it is created under {DE_HOME} directory.

- Open the {DE_HOME}/conf/de-config/application.properties file in a text editor.
- Add the following properties to the file. Replace the ${filesystem_loc} placeholder with your actual {filesystem} path:
  - localRepo=${filesystem_loc}/rules-repo
  - temporaryFolder=${filesystem_loc}/temp
  - decision.de.export.directory=${filesystem_loc}/export
- Add spring.profiles.active=fs-repository

For advanced security configuration, refer to the Security Configuration section.

5.3.5.2 DE Server – Starting and Stopping

Note: Before starting the server, you may validate if all configurations are correct by running **version.sh**.

For example, {TOMCAT_HOME}/bin/version.sh

To start the DE server:

- Run {TOMCAT_HOME}/bin/startup.sh

To stop the Decision server:

- Run {TOMCAT_HOME}/bin/shutdown.sh

Note: To verify the server is up with no ERROR in the process, use the following curl:

(Subject to actual context definition, here assuming default “/de”)

For example, curl http://<host>:<port>/de/ws/rs/api/actuator/health

Any connection failure at this point might relate to wrong port, wrong IP or DNS, Firewall block etc.

Check DE logs at {DE_HOME}/log and Tomcat logs at {TOMCAT_HOME}/logs for more info.

5.3.6 Upgrade Procedure

To upgrade DE to the current version, perform the following:

1. Stop the server.
2. Backup your data, database, or file system repository.
3. Backup the BDES directory and bdes.war file in {tomcat_home}/webapps.
4. Delete the BDES directory and bdes.war file from {tomcat_home}/webapps.
5. Copy the new bdes.war file from the release package location to {tomcat_home}/webapps.
6. In {de_home}/conf/de-config/application.properties, set decision.de.db.repository.monitor.disabled as 'true' in case of DM internal DE as shown below:

- decision.de.db.repository.monitor.disabled=true

Note: From DE 7.1.2 release onwards, there is no need for any manual changes on DE database schema.

7. Start the upgraded DE server and review the DE logs to verify that there are no errors. Schema changes for current version will be applied automatically and audit of these changes can be viewed in DE log or in DE new database table 'flyway_schema_history'.
8. Once DE application is up and running, the connection test can be done using the HTTP browser or curl.

HTTP Request Example: Curl http://<host>:<port>/de/ws/rs/api/actuator/health

Step 1: Set Up Global Class-Path and Configuration Folders

1. **Edit the Tomcat Global Configuration:**

- Locate and open the catalina.properties file in the <Tomcat folder>/conf directory.
- Set the shared.loader property to include the Decision configuration directory:
  - shared.loader=${catalina.home}/de-config

2. **Create Configuration Directory:**

- If it does not already exist, create a new directory named de-config in the Tomcat root directory.

3. **Create Configuration Sub-Directories for Each Application:**

- Inside the decision-config directory, create the following sub-directories for each Decision application:
  - de-config
  - PM (for PaM)
  - health-service-config

4. Add Configuration Files:

   In each sub-directory, create a configuration file named either application.properties or application.yml, based on your preferred format. These files will be used to define the properties of each application, as detailed in the Configuration section.

Step 2: Configure Application Contexts in Tomcat

1. **Edit the Tomcat's server.xml File:**

- Locate and open the server.xml file in the <Tomcat folder>/conf directory.

- Within the <Host> tag, add <Context> entries for each Decision application, as shown below:

```
<Host
name="localhost" appBase="webapps" unpackWARs="true" autoDeploy="true"
startStopThreads="10">
<Context docBase="bdes" path="/bdes"/>
<Context docBase="pam" path="/pam"/>
<Context docBase="pam-ui" path="/pam-ui"/>
<Context docBase="health-service" path="/health-service"/>
</Host>
```

2. **Adjust Start-Stop Threads:**

- Set the startStopThreads attribute based on the number of <Context> entries.
- The recommended value is the total number of applications you are deploying.

Step 3: Configure Tomcat HTTP and Shutdown Ports

1. Set HTTP Port:

• In the <Connector> tag, set the HTTP port (default is 8080):

```
<Connector port="9080" protocol="HTTP/1.1"
connectionTimeout="20000"
redirectPort="8443"
maxParameterCount="1000" />
```

2. Set Shutdown Port:

- Change the shutdown port in the <Server> tag (default is 8005) to avoid conflicts:

```
<Server port="9005" shutdown="SHUTDOWN">
```

Step 4: Create and Configure Tomcat setenv.sh Script

1. **Create the setenv.sh Script:**

- If not already present, create a setenv.sh file in the <Tomcat folder>/bin directory.

2. **Add Environment Variables and Java Options:**

- Add the following content to the setenv.sh file:

```
#!/bin/sh
export JAVA_HOME="/home/.jdks/corretto-17.0.12"
export JAVA_OPTS="-Xmx8g \
-Ddecision.home=${catalina.home} \
-Ddecision.de.home=${catalina.home} \
-Dsun.io.useCanonCaches=true \
-XX:+UseParallelGC \
-Dfile.encoding=UTF-8 \
-Dcom.sun.net.ssl.enableECC=true \
--add-opens=java.base/sun.reflect.annotation=ALL-UNNAMED \
--add-modules=java.se \
--add-exports=java.base/jdk.internal.ref=ALL-UNNAMED \
--add-opens=java.base/sun.nio.ch=ALL-UNNAMED \
--add-opens=java.base/java.lang=ALL-UNNAMED \
--add-opens=java.management/sun.management=ALL-UNNAMED \
--add-opens=jdk.management/com.sun.management.internal=ALL-UNNAMED \
--add-opens=java.base/java.util=ALL-UNNAMED"
```

3. **Edit Configuration:**

- Set the `JAVA_HOME` variable to your Java 17 JDK installation path.
- Adjust the `-Xmx4g` value (maximum memory) based on your system specifications.

Step 5: Prepare Database Schemas

- Ensure that the database schemas for each Decision application (DT, PaM, and Health Service) are ready.
- These schemas must have the necessary permissions (read/write) as specified in the Prerequisites section.

Step 6: Deploy Application WAR Files

1. **Stop Any Existing Decision Applications:**

- If any Decision applications are running in the target Tomcat server, stop the server.

2. **Copy WAR Files:**

- Copy the Decision application WAR files to the `<Tomcat folder>/webapps` directory.

3. **Rename WAR Files:**

- Remove the version numbers from the WAR filenames:
  - de-7.5.2.war → bdes.war
  - pam-1.5.4.war → pam.war
  - pam-ui-1.3.war → pam-ui.war

4. **Clean Up Existing Files:**

- Delete any existing WAR files or folders of a Decision application you are about to redeploy.

Step 7: Configure Application Properties

- Follow the Configuration instructions to set the required properties for each application under the "All-in-One" section.
- Populate the configuration files (`application.properties` or `application.yml`) created in Step 1 with these properties.

5.4 Standalone Deployment

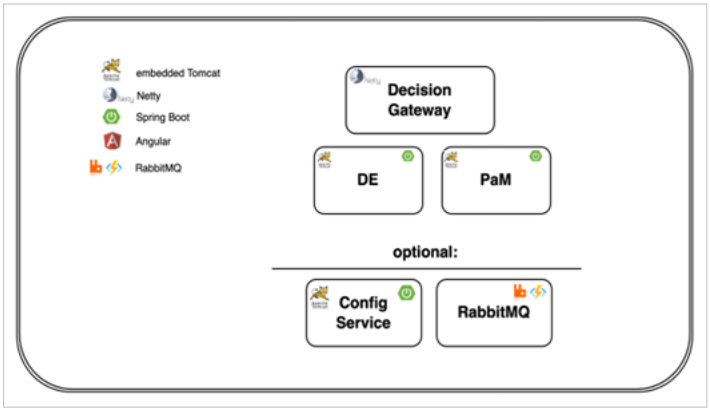
5.4.1 Overview

In standalone deployment, each Decision application is packaged as a self-executable JAR file generated using the Spring Boot framework. These JAR files contain an embedded Tomcat server, making them completely independent and self-sufficient.

- **Independent Execution:** Each application runs on its own port and can be hosted on the same or different machines.
- **Application-Specific Configuration:** Server-related properties (e.g., port, SSL) are managed directly in the application's properties file (`application.properties` or `application.yml`) or via Java command-line arguments.
- **Improved Flexibility and Scalability:** This model allows fine-grained control over resource allocation, monitoring, and scaling for each application.

Advantages:

- Distributed points of failure — Failure of one component does not affect others.
- Maximum flexibility, scalability, and isolation between application components.
- Fast startup due to isolated application instances.
- Independent memory and resource allocation for each component.
- Per-application monitoring using independent Java processes.
- Fine-grained scaling for each application.



Standalone Deployment Architecture

5.4.2 Package Content

The package for standalone deployment includes the following components:

- **Application Java Binary Files:** Compiled Java class files for each Decision application.
- **Spring Boot Framework Files:** Essential Spring Boot classes for server startup and management.
- **Java Third-Party Libraries:** JAR files of external libraries used by the applications.
- **Maven Build Metadata:** Details of build configuration and dependencies.

5.4.3 Recommended Use Cases

Standalone deployment is ideal in the following scenarios:

- When you prefer to build your own container images using the application JAR files.
- When fine-grained scalability, resource allocation, monitoring, and isolation for each application are required.
- When fast recovery and startup for each application is essential.

Common Use Cases:

- Production environments with a large number of users.
- High-traffic scenarios with frequent server requests.

5.4.4 Prerequisites

The standalone deployment of Decision requires specific platform dependencies for each component to ensure optimal performance and smooth operation. These prerequisites include Java installation, available disk space, memory allocation, database setup, and connectivity between components.

5.4.4.1 Java Installation (Global Requirement)

- **JDK Version:** Installed JDK distribution of Java 17.
- **JAVA_HOME Configuration:** Ensure the Java installation path is set in the JAVA_HOME environment variable, either at the OS level or through the application's Java command line.
- **Memory Allocation (Xmx):** The memory allocation size for each application is determined by the expected user load and concurrent request count. It can be set via Java command line arguments. For example:

```
java -Xmx1g -jar <app_jar_file_path>
```

- **Disk Space (Minimum Requirements):** Each application has specific disk space requirements listed below.

.

5.4.4.2 Component-Specific Prerequisites

Component	Disk Space	Memory (Xmx)	Additional Dependencies
DE	185 MB	Minimum: 1.4 GB	Dedicated database schema (PostgreSQL/Oracle). Adjust memory based on DE asset artifacts cached.
PaM	120 MB	Min: 400 MB, Rec: 800 MB	Dedicated database schema (PostgreSQL/Oracle).

Component	Disk Space	Memory (Xmx)	Additional Dependencies
Decision Gateway (OAuth)	75 MB	Minimum: 500 MB	Redis Database. OAuth IDP (e.g., Keycloak). Connectivity to other Decision applications.
Configuration Service (Optional)	75 MB	Minimum: 425 MB	RabbitMQ (version 4.0.7 and above).

.

5.4.5 Installation

To install any Decision application in standalone mode:

1. **Database Schema Setup**
   - If the application requires a database schema, ensure it is created.
   - If the schema does not exist, create it before proceeding.
2. **Application Jar Placement**
   - Copy the self-executable JAR file of the application to your desired application home folder.
3. **Configuration File Setup**
   - Create or copy an `application.properties` or `application.yml` file.
   - Ensure this configuration file is in the same folder as the application JAR file.
4. **Configuration Settings**
   - Refer to the **Configuration** section.
   - Populate the configuration file with the required properties under the **Standalone** sections.

5.5 Post Installation Validation

5.5.1 Health Check Validation

The following new endpoints were introduced to perform heartbeat check and health check in DE:

- https://<host>:<port>/de/ws/rs/api/heartbeat

  For example, `http://localhost:9080/de/ws/rs/api/heartbeat`
- https://<host>:<port>/de/ws/rs/api/2_0/heartbeat

  For example, `http://localhost:9080/de/ws/rs/api/2_0/heartbeat`

The heartbeat API returns the service version.

Response example:

```
{"version":"7.1"}
https://<host>:<port>/de/ws/rs/api/actuator/health


https://<host>:<port>/de/ws/rs/api/2_0/actuator/health


The health API returns the service health indicators.
Response example:
{
"database": {
"status": "UP",
"details": {
"time": 2
}
},
"serviceDetails": {
"status": "UP",
"details": {
"serviceName": "de",
"version": "7.1",
"time": 0
}
}
}
```

The following are the DE Health Indicators:

- **database:** Indicates the Database connectivity.
- **serviceDetails:** A special indicator that returns the version and the name of the service.

5.5.2 DE Startup Process and Logs

Depending on the actual setup, DE application will start as part of Tomcat initial process.

As soon as DE application starts, DE logs files will be created in the location set by the variable “logging.file.path”. Additional log files will be found in {TOMCAT_HOME}/logs directory. The main DE log file is “bdes.log” which will include most of the information along with some in additional files.

DE log file entries to start with, depending on the actual setup:

`INFO [ContextLoader]Root WebApplicationContext: initialization started`

DB connection will issue the following message:

`[c.z.h.HikariDataSource] [] HikariPool-1 - Starting..`

The initial setup process will finish with the following message:

`INFO [ContextLoader] Root WebApplicationContext: initialization completed in xxx ms`

5.5.3 Default DE Timezone for logic version date determination

Currently, when requests are made to DE with the `executablekey` and `effectiveDate` field without a timezone, the default timezone used for the date is the "server" timezone.

When the timezone is not provided, a new custom property `bdes.default.timezone` is introduced to define the DE timezone for logic version date determination.

When DE is called with an effective date, it will be based on the timezone date that the custom property is set in. The value for the custom property can be "server", "GMT" or "GMT+/-n" where "server" is the default value with the following conditions:

- When the property is set to "server" then it should display the current server time zone.
- When property is set to "GMT" then time will be "00:00:00".
- When property is set to "GMT+/-n" then the displayed time should be based on the given time zone. The GMT hour values would be between the range of -11 to +12 and the valid value for minutes are 0 or 30 only but with an exception value which is 5:45.

  For example, GMT+5:30, GMT-3:00, GMT+4.

Note: If the time zone is not available and the property value is not a valid time zone, then the default "server" option will be considered.

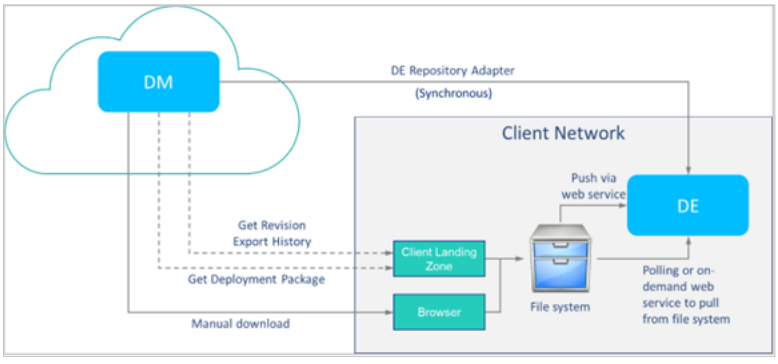
5.5.4 Managing Deployments across Hosted and Client Environments from DM to DE

5.5.4.1 Remote DE Deployment Overview

Decision Manager (DM) has a built-in Repository Adapter for deploying Project releases to Decision Execution (DE). This works seamlessly when DM and DE are configured in the same environments or in environments that have access to one another. This is the case for DM and DE, which are managed in the Sapiens cloud by Sapiens Cloud Services (SCS). For some clients, Decision Manager (DM) will be hosted on a cloud environment; however, Decision Execution (DE) will be hosted in a client environment due to Decision execution security or performance requirements.

The diagram below shows the multiple options for DM to DE deployment, both connected and remote. For connected deployments, DM will automatically communicate with DE and manage the deployment. For Remote deployments, the deployment “Zip” file will need to be obtained. The deployment “Zip” file can be obtained via a manual download of a Project Release from the Decision Manager, automated via the Decision User Interface via RPA, or via the use of the Decision Manager web services “Get Revision Export History” to obtain a list of exported Project Releases ready for download and the “Get Deployment Package” to download each exported Project Release.

For more information on the “Get Revision Export History” and “Get Deployment Package” web services, refer to the *Decision Manager (DM) Client Integration APIs* guide, REST Web Service APIs, and DevOps Services.



5.5.4.2 DE Repository Adapter

When DM has access to DE web services for deployment, either because it is hosted by Sapiens or a client can expose their locally installed DE services to the internet then the DE Repository Adapter can be used. The DE environment can be configured in DM as an environment and the DE Repository Adapter “Decision Execution (DE)” can be selected for the configured environment. The Decision Repository adapter supports multiple authentication types, unsecured, Basic Authentication, and OAuth 2.0.

The DE Repository Adapter is the most seamless and out-of-the-box deployment option and should be utilized if possible.

5.5.4.3 DE Web Service

DE has a web service `InvokeRemoteDeployment` that can be called to deploy a Project release “Zip” file to DE. This service can be used if your process will be to obtain the Project release “Zip” file and not store it locally on a file system, but just deploy it to DE.

Note that with this approach, there may be the possibility of a web service timeout for extremely large Project release packages.

To send the Project Release package to DE via web service, send a POST request to the following address:

- `{DE_ADDRESS}:{PORT}/de/ws/rs/remote/deployment/deployArtifacts`

Note: The request should include a parameter "zipFile" which is the Project release “Zip” file to deploy to DE. This is encoded in Base64 format.

A sample Java program to deploy a saved project release “Zip” file to DE with Basic Authentication is given below:

```
package com.sapiens.decision;
import java.io.IOException;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Base64;
public class deployToDeConsoleApp {
/* args[0] - zip path
args[1] - rest url
args[2] - user
args[3] - password (plain text)
*/
public static void main(String[] args){
String pathToZip = args[0];
String restURL = args[1];
String user = "";
String password = "";
if (args.length == 4) {
user = args[2];
password = args[3];
}
byte[] zipFileAsByte = getByteArrayFromFile(pathToZip);
if (zipFileAsByte != null) {
deployToDE(restURL, user, password, zipFileAsByte);
}
return;
}
private static void deployToDE(String restURL, String user, String
password, byte[] zipFileAsByte) {
URL url;
HttpURLConnection conn;
OutputStream os = null;
try {
url = new URL(restURL);
conn = (HttpURLConnection) url.openConnection();
conn.setDoOutput(true);
conn.setRequestMethod("POST");

if (!user.isEmpty() && !password.isEmpty()) {
String authString = user + ":" + password;
String encodedAuthString = Base64.getEncoder().encodeToString
(authString.getBytes());
conn.setRequestProperty("Authorization", "Basic " + encodedAuthString);
}

conn.setRequestProperty("Content-Type", "application/octet-stream");
conn.setRequestProperty("Content-Length", String.valueOf
(zipFileAsByte.length));
os = conn.getOutputStream();
os.write(zipFileAsByte);
os.flush();
if (conn.getResponseCode() == 204) {
System.out.println("File was successfully deployed to DE");
} else {
System.out.println("Failed to deploy to DE. HTTP error code: " +
conn.getResponseCode());
}
} catch (IOException ioException){
System.out.println("Exception occurred while deploying to DE: " +
ioException.getMessage());
} finally {
```

```
try {
os.close();
} catch (IOException e) {
System.out.println("Exception occurred while closing the connection: " +
e.getMessage());
}
}
}
private static byte[] getByteArrayFromFile(String pathToZip)  {
Path path = Paths.get(pathToZip);
byte[] data = null;
try {
data = Files.readAllBytes(path);
byte[] encStr = Base64.getEncoder().encode(data);
return encStr;
} catch (IOException e) {
System.out.println("Can't read file from path:" + path);
return null;
}
}
}
```

5.5.4.4 DE File System and Notification Event

The DE file system and notification event approach to a remote deployment assumes that the DM Project Release “Zip” file will be downloaded from DM and saved to a file location in the client environment. The file location can be configured in DE for DE to obtain a file for Deployment. This process is useful if you have a file location to place Project Release “Zip” files, but do not want to wait for a polling frequency to complete the deployment, but would rather trigger DE to trigger a deployment immediately.

With a source location in a File System for the Project release “Zip” file, the DE web service will investigate the File System and fetch all the files to deploy into DE currently in the file system. The service locks the processing of a file and will only process files that are not locked. This is to ensure that different nodes of DE do not process the same import file.

To manage deployments across the hosted and client environments of DM to DE, perform the following:

Deploying from a location

1. Edit application.propertiesfile and add the following properties:
   - decision.de.remote.deployment.location.path={path_to_folder_with_deployments}
   - decision.de.remote.deployment.location.type={location_type}

Note: Currently only file_system is supported.

   - decision.de.enable.remote.deployment=true
2. Trigger a REST service for DE to deploy all Project release “Zip” files to DE.

   Send a POST request to the following address:

   {DE_ADDRESS}:{PORT}/de/ws/rs/remote/deployment/deployNewArtifacts

   without parameters

   For example, http://localhost:9080/de/ws/rs/remote/deployment/deployNewArtifacts

5.5.4.5 DE file system and polling for new deployments

The DE file system and polling for new deployments are similar to the DE file system and notification event process, except that the notification event to trigger DE to deploy files is replaced with a DE configuration to automatically poll the file system for new deployments. This approach is useful if you have a process to move the Project release “Zip” file to a file location and would not require any additional process development.

To manage deployments across the hosted and client environments of DM to DE, perform the following:

Deploying from a location

1. Edit the application.properties file and add the following properties:

- decision.de.remote.deployment.location.path={path_to_folder_with_deployments}
- decision.de.remote.deployment.location.type={location_type}

Note: Currently, only file_system is supported.

- decision.de.enable.remote.deployment=true

2. To perform Auto Polling from the location, edit the application.properties file and add the following properties:

- spring.profiles.active=remote-deployment-polling
- decision.de.remote.deployment.frequency.seconds={num_of_seconds_to_poll}

Note: If you are using a cluster configuration or environment, a database repository is needed to synchronize the different instances in the Cluster.

6. Database Implementation

The following implementation is for unencrypted access. For encrypted access details, refer to the Property Value Encryption section.

6.1 Oracle

1. Edit application.properties.
2. Define a user or use an existing one (such as 'DecSQLUser' shown in the example image below).
3. Add following properties in application.properties:

- spring.datasource.url: Add the JDBC URL to the database.
- spring.datasource.username: Add the database username.
- spring.datasource.password: Add the database user password.

File example:

- spring.jpa.hibernate.ddl-auto=validate
- spring.datasource.url=jdbc:oracle:thin:@127.0.0.1:1521:orcl
- spring.datasource.username=desuser
- spring.datasource.password=despass1

4. Create a DE schema in Oracle database.

6.2 MS-SQL

1. Edit application.properties.
2. Define a user or use an existing one (such as 'DecSQLUser' shown in the example image below).
3. Modify by replacing the following placeholders:

- spring.datasource.url: Add the JDBC URL to the database.
- spring.datasource.username: Add the database username.
- spring.datasource.password: Add the database user password.

File example:

- spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=desdb;
- spring.datasource.username=desuser
- spring.datasource.password=despass1

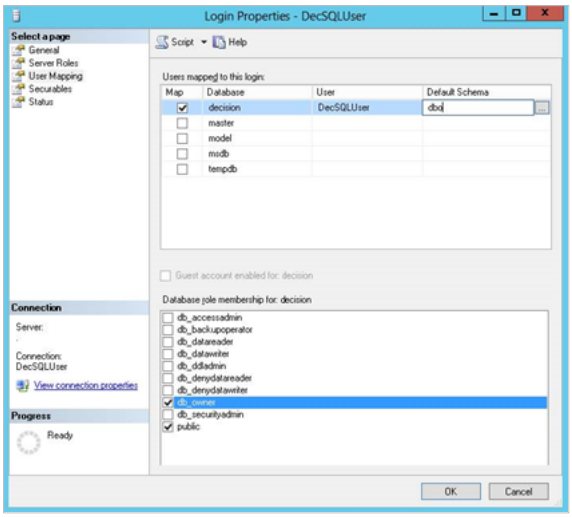
4. Create the database.
5. Navigate to **Security > Login**, right-click on the user and select **Properties**. Amend the following user's properties (see example below):

a. General tab: Change the user default database to the DE database.

b. User Mapping tab:

- Check map with database name. Change default schema to dbo.
- Check database owner.

c. Click OK.



6. Run the following commands:

- ALTER DATABASE decision SET READ_COMMITTED_SNAPSHOT ONALTER DATABASE decision SET ALLOW_SNAPSHOT_ISOLATION ON

7. Create a DE schema in the MS-SQL database.

6.3 PostgreSQL

1. Edit application.properties.
2. Define a user or use an existing one (such as 'DecPostgresUser').
3. Modify by replacing the following placeholders:

- spring.datasource.url: Add the JDBC URL to the database.
- spring.datasource.username: Add the database username.
- spring.datasource.password: Add the database user password.

File example:

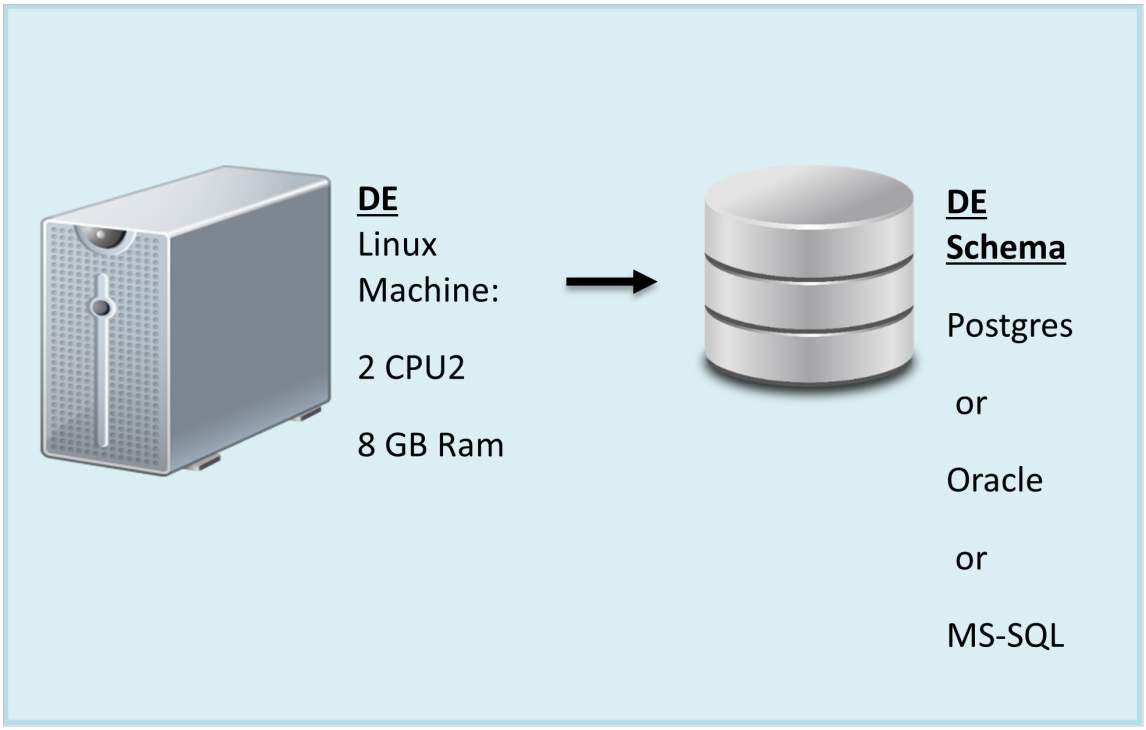
- spring.datasource.url=jdbc:postgresql://localhost:5432/{databaseName}?currentSchema=bdes
- spring.datasource.username=DesPostgresUser
- spring.datasource.password=DesPostgresPass1

4. Create the database.
5. Create a DE schema in PostgreSQL database

7. Installation Section

7.1 DE Installation Overview

The following sections detail the DE installation procedure. The same installation may serve as a DM internal DE application and as part of the DM installation.



DE – Single Instance Setup

8. Advanced Configuration Topics

8.1 Tomcat Datasource Configuration

The following instructions are for embedded Tomcat data source configuration in WAR and bundled installations.

1. Define the data source in Tomcat's context.xml file.

   Add a <Resource> tag in the context.xml file. Note that each data source has its own properties, and the attributes specified in the <Resource> tag should be made according to the specific data source.

   For example, the standard data source javax.sql.DataSource can be configured according to the specifications, while the data source used in Decision should be configured with the type com.mchange.v2.c3p0.ComboPooledDataSource.

2. Add the following lines to `application.properties`:

- Append spring.active.profiles with jndi-datasource
- decision.de.datasource.jndi.name=jdbc/TestDB

The jdbc/TestDB parameter should be the JNDI name for the datasource as configured in the tomcat.

8.2 Security Configuration

8.2.1 Basic Authentication

To configure the system as unsecured:

1. Open `application.properties` and set/append "spring.active.profiles" property with unsecured, so the property will be:

- spring.active.profiles=unsecured

Note: The `spring.active.profiles` property value needs to be changed, and the given value is only used as default.

To configure the system as secured:

1. Open `application.properties` and set/append "spring.active.profiles" property with secured, so the property will be:

- spring.active.profiles=secured

2. Generate an encrypted password using the decision encryptor. Refer to Property Value Encryption to know more about password encryption or other fields.
3. In the `security-users.properties` file located in the secured folder `DECISION_HOME/conf/de-config/`, define users and their passwords for authentication and grant them appropriate permissions.

Add user permissions for each user, as follows:

- username=<BCrypt_encrypted_ password>,permission1,permission2,permission3…,grantaccess[enabled/disabled]

For example,

“admin= $2a$10$Nkl8/KFaoy1x….CSt7EaiCVDyeItZ1tjJyTXGi,ROLE_USER,ROLE_ADMIN, ROLE_ MONITORING, enabled”

where:

- **admin**: Defines the username.
- **$2a$10$Nkl8/KF….iCVDyeItZ1tjJyTXGi**: Defines the password encrypted by BCrypt hash generated as described in Property Value Encryption.
- **ROLE_USER**: Defines the authorized roles (separated by commas) for users that can execute Decisions Flows and Decisions. ROLE_USER can also execute services used in Decision Execution such as “Get Latest tag” and “Get Manifest”.
- **ROLE_ADMIN**: Defines the authorized roles (separated by commas) for users that can execute Administrative services such as deploying and undeploying Decisions Flows and Decisions to DE.
- **ROLE_MONITORING**: Provides access to DE Monitoring.
- **enabled**: Defines the permission mode (which can be enabled or disabled).

Permissions by Role

DE Service	ROLE_ADMIN	ROLE_USER
DeIndicatorWebService		
checkDEConnectionAndCheckCredential	Y	Y
ExecutionWebService		
executeFlowAsynchronous	N	Y
executeAsynchronous	N	Y
executeFlow	N	Y
findAllArtifactKeys	Y	Y
findLatestTag	Y	Y
batchExecuteFlow	N	Y
deleteArtifact	Y	N
findKnowledgeBaseContent	Y	Y
checkHeartBeat	Y	Y
findAllDeploymentAudit	Y	Y
batchDecisionExecute	N	Y
batchExecuteFlowAsynchronous	N	Y

DE Service	ROLE_ADMIN	ROLE_USER
saveArtifact	Y	N
saveArtifacts	Y	N
getDecisionManifest	Y	Y
batchDecisionExecuteAsynchronous	N	Y
deleteAllArtifactsInTag	Y	N
findAllArtifactKeysWithVersionAndEffectiveDates	Y	Y
execute	N	Y

RepositoryWebService	ROLE_ADMIN	ROLE_USER
findAllArtifactKeysWithVersionAndEffectiveDates	Y	Y
exportRepository	Y	N
findAllArtifactKeys	Y	Y
findLatestTag	Y	Y
exportRelease	Y	N
exportTag	Y	N
importAndGenerateInputWithExpectedResult	Y	N
exportArtifact	Y	N
findAllDeploymentAudit	Y	Y

SimpleExecutionService	ROLE_ADMIN	ROLE_USER
execute	N	Y
executeFlow	N	Y
executeFlowWithCallback	N	Y
executeWithCallback	N	Y

REST endpoints(for functions except the above)	ROLE_ADMIN	ROLE_USER
/ws/rs/api/*/execute/di/**	N	Y
/ws/rs/remote/deployment/**	Y	N
/ws/rs/api/?.*/execute/heartbeat	Y	Y
/ws/rs/api/?.*/execute/serverstatus	Y	Y

4. To configure the system for password encryption:

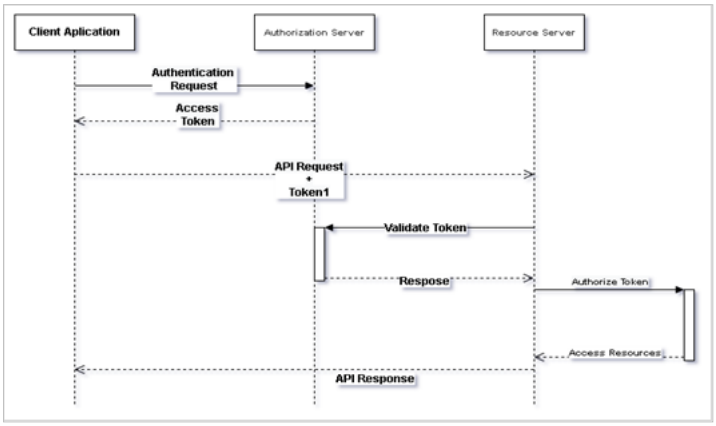
Refer to the Property Value encryption to know more about password encryption or other fields.

Note: If the decryption of any of the encrypted properties fails, the error is logged with the name of the property, and the application startup fails.

8.2.2 OAuth 2.0 Authentication

OAuth is a standard designed for online authorization that allows a website or application to access resources hosted by other web apps on behalf of a user.

To integrate OAuth 2.0 with DE, you must do the following:



8.2.2.1 OAuth 2.0 Configuration

To configure DE for OAuth 2.0 authentication, use the following properties in the application.properties file:

```
spring.profiles.active=oauth2
spring.security.oauth2.resourceserver.jwt.audiences=https://de-for-testing
spring.security.oauth2.resourceserver.jwt.issuer-uri=<IDP issuer URL>
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=<IDP jwk set-uri
URL>
```

Note: The jwk-set-uri property is optional but recommended for enhanced security.

1. To successfully access the resource endpoint (API) it is mandatory to add at least one of the following scopes in the JWT token:

- DE_USER
- DE_ADMIN
- DE_MONITORING

The following table describes the mapping of scopes to roles:

Scope	Role
DE_USER	ROLE_USER
DE_ADMIN	ROLE_ADMIN
DE_MONITORING	ROLE_MONITORING

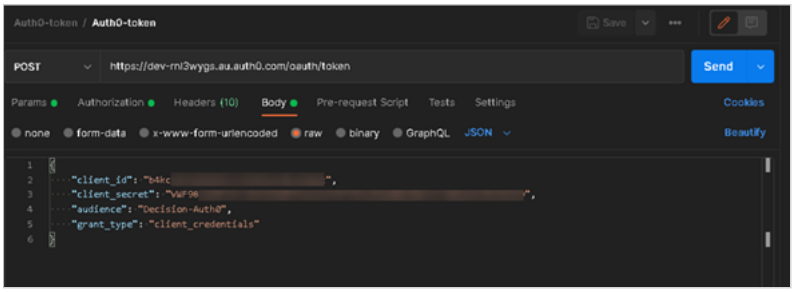
Refer to the Basic Authentication section to learn about the definition of the roles.

8.2.2.2 Invoke Client API's using Access Token

To use OAuth 2.0 when making web service requests, in the request header you will need to specify the "OAuth 2.0" authorization type and add the appropriate access token obtained from the OAuth 2.0 provider.

Get Token

An example of how to obtain an access token for Auth0 using Postman is given below:

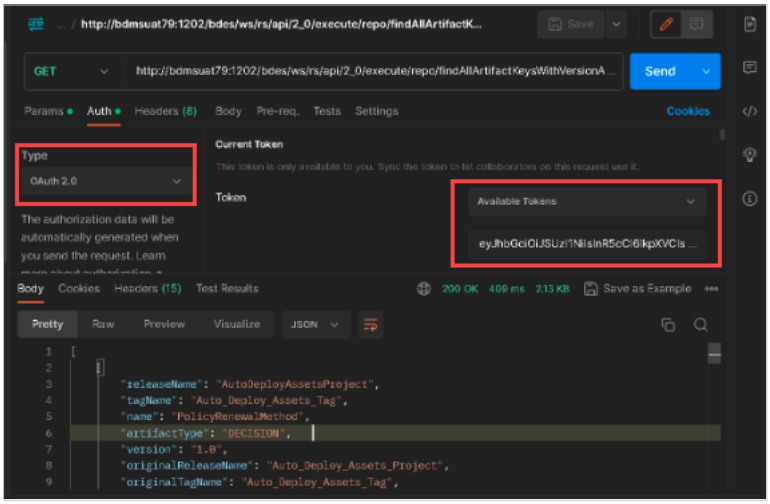


Obtaining Access token using Postman

API Request

To invoke the API's using the access token in Postman:

1. Navigate to the **Authorization** tab.
2. Select the type as **OAuth 2.0**.
3. In the **Access Token** field, add the generated/obtained access token.



Invoke API using access token

8.3 Property Value Encryption

8.3.1 Overview

Decision applications allow secure storage of sensitive configuration values (such as database passwords, API keys, and credentials) by supporting encrypted properties in the configuration

files (application.yml or application.properties). Encrypted property values enhance security by ensuring that sensitive information is protected even if the configuration files are exposed.

8.3.2 How Property Encryption Works

- Properties marked as encrypted use the prefix ENC: to indicate they are securely encrypted.
- The encrypted value must be generated using the **Decision Encryption Tool**, which applies an RSA encryption algorithm.
- During runtime, the Decision application automatically decrypts these properties using the corresponding decryption key.

Example:

```
# Encrypted database password in application.properties
spring.datasource.password=ENC:lmclyieQY87EJD.....
```

8.3.3 Encryption Tool Usage

The **Decision Encryption Tool** (decision-encrypt-2.0.jar) is a command-line utility that helps you generate encrypted values using RSA encryption.

Supported Algorithms:

- RSA (Recommended)

8.3.4 Generating Encrypted Property Values

1. **Ensure Java is Installed**
   - Make sure JDK 17 or above is installed and configured on your system.
   - Set the JAVA_HOME environment variable if not already set:

```
export JAVA_HOME="/path/to/jdk-17"
```

2. **Obtain the Encryption Tool**
   - Ensure that the decision-encrypt-2.0.jar file is available on your system.
3. **Run the Encryption Tool**
   - Use the command-line interface (CMD on Windows, Terminal on Linux) to run the tool.

Windows Command Example:

```
java -jar decision-encrypt-2.0.jar RSA value-to-encrypt
```

Linux Command Example:

```
java -jar decision-encrypt-2.0.jar 'RSA' 'value-to-encrypt'
```

- Replace value-to-encrypt with the actual sensitive value you want to secure (e.g., a database password).

4. **Review the Tool Output**

- The tool will output the following information:

```
Entered Text - (The value you entered for encryption)
Encryption Public Key - (Only for reference)
Encrypted text - (The encrypted value to be used in your configuration
file)
```

- Copy the Encrypted Text value.

5. **Add Encrypted Value to Configuration**

- Place the encrypted value in the configuration file (application.yml or application.properties) with the prefix `ENC:`.

Example for application.yml:

```
spring:
datasource:
url: jdbc:postgresql://localhost:5432/decision_de
username: de_user
password: ENC:lmclyieQY87EJD.....  # Encrypted password
```

Example for application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/decision_de
spring.datasource.username=de_user
spring.datasource.password=ENC:lmclyieQY87EJD.....
```

6. **Verify Decryption During Application Startup**

- Start the Decision application.
- Check the application logs to ensure there are no decryption errors.
- The application should automatically decrypt the value at runtime.

8.4 Allowlisting Knowledge Models

To mitigate security risks associated with executing externally provided Knowledge Model web services, these services must be allowlisted as a security control. This measure ensures that only pre-defined services can be executed within a decision. If a Knowledge Model URL is not on the allowlist, the Decision will not execute it and will return an exception. To minimize the need for allowlisting individual Knowledge Model web service URLs, URLs can be defined at higher levels to encompass multiple web services within the same namespace.

The following property allows you to enable the allowlisting of a Knowledge Model:

- decision.de.km.security.allowed-host

This property specifies a comma-separated list of rules that define the allowed Knowledge Model (KM) web service URLs. Each rule consists of a host and port combination, with an optional path. You can use an asterisk (*) as a wildcard character for the host, port, or both.

Note: If no value is provided for this property, all Knowledge Model service calls are blocked by default.

To allow unrestricted access to all Knowledge Model URLs:

- decision.de.km.security.allowed-host=*:*

Note: In the YAML configuration file, all values must be enclosed in quotation marks.

For finer control, list the allowed host, port, and path combinations as a comma-separated list. Wildcards are supported.

- decision.de.km.security.allowed-host=192.168.0.1:*,localhost:8080,*.sapiens.com:*,10.0.0.1:*/api/

8.5 Customized Configuration Properties

Properties in DE are controls that can be set to determine the way the system behaves.

Customized properties are defined in several dedicated files that DE reads and implements upon startup.

This section describes customized files and properties that can be used by system administrators to control the behavior of DE components. Only authorized system personnel should modify the contents of the property files.

The files and properties described in this document are not associated with a specific DE release. Some properties will work with a specific release while others may not.

Configuration Property File List

The following files include customized configuration properties used by DE.

Click a file name for an explanation of the file and the properties it contains.

8.5.1 application.properties

The application.properties file allows users to override default property values in DE.

Name	Valid Values	Default value	Description
decision.de.db.repository.monitor.disabled	Boolean (true/false)	false	Defines the enabling or disabling of an automated polling process in DE to check the DE repository for new deployments and automatically load the new deployment into the DE cache for execution. This property should be set to false to not disable the automatic polling when DE is used in a production execution environment. This ensures that deployments to a production environment are automatically loaded to DE for immediate execution availability. This property should be set to true when DE is used for the testing facility of DM. For DM, assets are deployed one at a time and automatically executed. Turning the polling off will ensure no conflicts between the polling and the executing process loading the same asset in the cache, preventing situations where the execution will start before the asset is loaded into cache.
decision.de.execution.conclusion.factType.cache.size	Numeric	500000	Defines the size of the cache that holds the mapping of executable key and the associated Fact Types.

Name	Valid Values	Default value	Description
bdes.execution.input.must.belong.to.factTypes.in.manifest.request.override	String (Null, True, and False)	Empty	Defines the validation of InputMustBelongToFactTypesInManifest in a request that can be altered or overridden by this property. You can provide any one of the following values while setting up the property: • True: Overrides setting in request message setting with 'true' for InputMustBelongToFactTypesInManifest. • False/ Invalid Values: Overrides setting in request message with 'false' for InputMustBelongToFactTypesInManifest. • Null/Empty: Uses the request message value for InputMustBelongToFactTypesInManifest.
decision.oauth2.resourceserver.jwt.scope.keys	String (An authorized client or user).	scope, scp	The property lists the JWT claim names from which the server extracts authorities (permissions or roles). For example, decision.oauth2.resourceserver.jwt.scope.keys=scope,scp **Note:** These claims map to application roles or authorities, defining the user's allowed actions.

Name	Valid Values	Default value	Description
decision.oauth2.resourceserver.jwt.authorized.party.keys	String (An authorized client or user).	sub,azp,client_id,cid	The property defines the keys in the JWT that identify the authorized party (client or user). It is used only to log the user who sent the request. For example, decision.oauth2.resourceserver.jwt.authorized.party.keys=sub,azp,client_id,cid **Note:** These keys allow the application to validate and identify the token's associated party.
decision.de.healthcheck.threadresources.active.threads.prefixes	You can provide a comma-separated list that replaces the default value. For example, tcp,c3p0,hz,reactor-tcp, reactor-http	http-nio	The property to review information on various active thread prefixes provided in the health check service response.

Batch Execution Concurrency Settings

Name	Valid Values	Default value	Description
decision.de.executor.core.pool.size	Numeric	10	Defines the minimum number of workers to keep alive without timing out for the asynchronous batch execution of Decisions and Flows.
decision.de.executor.max.pool.size	Numeric	20	Defines the max number of threads that can ever be created for the asynchronous batch execution.
decision.de.executor.queue.capacity	Numeric	50	Defines the capacity of the blocking queue that is used to hold and transfer the execution tasks for parallel processing of asynchronous batch execution.
decision.de.executor.keep.alive	Numeric (in seconds)	300	Defines the time limit for which threads may remain idle before being terminated.

Basic Authentication Account Lock Out Configurations

Name	Valid Values	Default value	Description
decision.de.authentication.failure.threshold	Numeric	3	If DE is running with basic authentication, this property defines the maximum number of invalid login attempts allowed before the account is locked out for a certain duration.
decision.de.authentication.user.locked.duration	Numeric (in minutes)	30	If DE is running with basic authentication, this property defines the duration for which the account is locked out if the user exceeds the maximum attempts allowed.

DB Pool Configurations

Name	Valid Values	Default value	Description
spring.datasource.hikari.maximum-pool-size	Integer	30	Defines the maximum number of connections a pool will maintain at any given time.
spring.datasource.hikari.minimum-idle	Integer	10	Defines the minimum number of connections a pool will maintain at any given time.

Name	Valid Values	Default value	Description
spring.datasource.hikari.idle-timeout	Integer (in milli seconds)	0	Defines the duration in seconds for c3p0 to test all idle, i.e. pooled but unchecked-out connections.
spring.datasource.hikari.max-lifetime	Integer (in milli seconds)	600000	Defines the time a connection can remain pooled but unused before being discarded. Zero means idle connections never expire.
spring.datasource.hikari.leak-detection-threshold	Integer (in milli seconds)	20000	If a connection is not returned within this threshold (ms), a stack trace is logged. Great for debugging.

DE Repository Monitor Configurations

Name	Valid Values	Default value	Description
decision.de.db.repository.monitor.disabled	Boolean (True/False)	0	Provides an option to disable the DE repository monitor if set to true.

DE Secured/Unsecured Configurations

Name	Valid Values	Default value	Description
spring.profiles.active=unsecured	TEXT (secured/unsecured)	unsecured	Provides an option to enable/disable DE secure mode.

8.6 Installing DE Gateway for Integration with Decision InfoHub (DI)

8.6.1 Configuring DE for Integration with DI

DE Custom Properties

The following DE Custom properties must be added to the DE Custom Property file (application.properties) to support integration with DI and to use the DE Gateway Execution service.

decision.de.infohub.server.host

This property is used to set the location of the InfoHub server. It shall be set to the host address.

Example: decision.de.infohub.server.host=127.0.0.1

decision.de.infohub.server.port

This property is used to set the port of the InfoHub server. It shall be set to the host server port.

Example: decision.de.infohub.server.port=3213

decision.de.infohub.server.ws.token

This property is used to set to the InfoHub security token. It shall be set to the host security token. This configuration will be overridden if the tokenId parameter is available in the input request.

Example: decision.de.infohub.server.ws.token=demo

decision.de.infohub.server.ws.name

This property is required and is used to set the name of the InfoHub Web Service that will be called by the DE Gateway to collect the data required for execution of the specified Decision or Decision Flow. DE is provided with an import file to import into DI the installation of the web service with the default name of “getAllLuDecisionInfo”.

Example: decision.de.infohub.server.ws.name=getAllLuDecisionInfo

decision.de.infohub.server.ws.instanceId.param

This property is used to define the DE Gateway Execution parameter name for the DI Logical Unit Id. It is defaulted to “instanceId” and if using the default DI “getAllLuDecisionInfo” will not need to be changed.

Example: decision.de.infohub.server.ws.instanceId.param=instanceId

decision.de.infohub.server.decision.completion.ws.name

Used in conjunction with the executeWithCallback method, this property is used to set the name of the InfoHub web service that will be called by the DE Gateway as a call back to DI after a DE Decision View execution. The DI call back web service provides the ability to execute DI logic on completion of the DE Decision execution for post processing logic. The full DE return message is provided to DI and all information in the message can be used for additional logic or processing. The call back web service also supports the ability to override the return XML message to the calling application. This property is defaulted to “onDecisionCompletion” and DE is provided with an import file to import into DI the installation of this web service with the default name. If the default DI Web Service is used this property should not be changed.

Note: If this property is not provided, the Callback service will not call InfoHub and will just return the standard DE return message.

Example: decision.de.infohub.server.decision.completion.ws.name=onDecisionCompletion

decision.de.infohub.server.flow.completion.ws.name

Used in conjunction with the executeFlowWithCallback method, this property is used to set the name of the InfoHub web service that will be called by the DE Gateway as a call back to DI after a DE Decision Flow execution. The DI call back web service provides the ability to execute DI logic on completion of the DE Decision Flow execution for post processing logic. The full DE return message is provided to DI and all information in the message can be used for additional logic or processing. The call back web service also supports the ability to override the return XML message to the calling application. This property is defaulted to "onFlowCompletion" and DE is provided with an import file to import into DI the installation of this web service with the default name. If the default DI Web Service is used this property should not be changed.

Note: If this property is not provided, the Callback service will not call InfoHub and will just return the standard DE return message.

Example: decision.de.infohub.server.flow.completion.ws.name=onFlowCompletion

decision.de.infohub.server.ws.secure.mode

Specifies whether the DE Gateway connects to DI using HTTPS or HTTP. Set the value to true to enable HTTPS, or false to use HTTP. The default value is false.

Example: decision.de.infohub.server.ws.secure.mode=false

8.6.2 Configuring DI for Integration with DE

Installing DI Web Services

Before the DE Gateway can integrate with DI, DI web services must be imported into DI. Each DE release will be included with a Decision InfoHub export file that will need to be imported for the release.

There will be two import files, as follows:

- DecisionLU_Services-1.0.k2export - Contains the DI web services to interact with DE and should be imported with each new version of DI.

  For example, for DE version 6.4 the DI Web Service import file will be called "DecisionLU_Services-6.4.k2export".

- DecisionLU_Callbacks-1.0.k2export - Contains DI web services template to configure callback code that can execute specialized logic specific to client implementations. This should only be imported upon initial DI installation as it will overlay any specialized logic if it is re-imported.

Importing an asset to InfoHub

To import the provided asset (DecisionLU_Services-1.0.k2export) to the InfoHub Studio, do the following:

1. Open the Infohub Studio.
2. In the Start Page, select New Project.
3. Give your project a name (such as InfoHub1324).
4. Right-click the project and select Import →Import All.
5. Select the provided asset (DecisionLU_Services-1.0.k2export).
6. Click Open > OK > Yes To All.

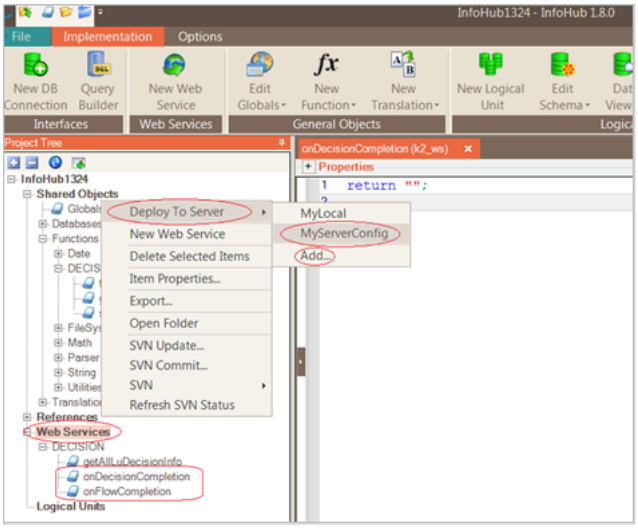
The project will be updated with all assets in the imported file.

For example, the Web Services branch (under the Project Tree view) should now contain the collection service “getAllLuDecisionInfo” and two CallBack services, “onDecisionCompletion” and “onFlowCompletion”.

The “getAllLuDecisionInfo” service is fully implemented and will not require any changes. The Callback services are provided as stubs and should be implemented to perform the Callback functionality desired and return an appropriate XML as a string in response or “null” (empty string) to return the standard DE execution response.

Deployment

After the above process is implemented and complete, the Web Services can be deployed to the selected Infohub Server, as shown in the following example.



Deployment

Servers can be added by clicking Add… (shown in the example above), or by selecting User Preferences in the Options menu at the top of the window.

8.6.3 Installing DE JARs for InfoHub

JARs and Installation Locations

The following JAR files must be installed in Decision InfoHub components as follows:

Studio:

- JAR Location: <installation folder>\InfoHub\classpath\(e.g. C:\Program Files (x86)\Sapiens Decision\InfoHub\classpath\)
- JARs (samples - will vary with each version of DE):
  - des-model-common-6.2-20160729.125315-13.jar
  - des-model-engine-6.2-20160729.125315-13.jar
  - des-model-execution-6.2-20160729.125315-13.jar
- Template location: <installation folder>\InfoHub\Config\

- Templates to update:
  - JavaClassTemplate.txt
  - JavaClassTemplateWS.txt

Server:

- JAR location:

  <server location>\server\fabric\driver\target\

or

<server location>\server\fabric\cassandra\lib\

- JARs (samples vary with each version of DE):

  Same 3 JARs used for Studio

8.7 Configuring Decision Flow and Decision View Schema in InfoHub

The DI “getAllLuDecisionData” is a generic web service that can return all required data to the DE Gateway in an XML message format used for DE execution. For this service to return the data in the DE Gateway required XML message format, there are restrictions on how a Decision View or Decision Flow Business Logical Unit (BLU) can be configured in InfoHub.

8.7.1 Naming a Business Logical Unit

The BLU name must be named in the following format for a Decision View and Decision Flow

Decision View

Conclusion: Policy Renewal Method

Version: 1.0

View: Base

LU Name = PolicyRenewalMethod__1_0__Base

Note: There must be two “_” between the conclusion and Version and Version and View name.

Decision Flow

DF Name: Policy Renewal Method Flow

Version: 1.1

LU Name = PolicyRenewalMethodFlow__1_1

Note: There must be two “_” between the conclusion and Version.

Important Note: From 6.13.1 onwards, BLU can be named anything as it has been decoupled from the name of the Asset and the same LU name should be passed in the request message for SimpleExecutionService endpoints.

8.7.2 Schema Tables and Table Names

The Schema shall have a table for each Repeating Group in the Decision View or Decision Flow model. Each Table shall be named “DECISION_<Repeating Group Name>” starting with the Root Repeating Group as “DECISION_ROOT” for all Fact Types in the root level of the Decision View or Decision Flow model. All Table Names shall be in uppercase and spaces replaced with an “_”.

If a DV or DF does not have any Repeating Groups then the BLU will have only a single “DECISION_ROOT”.

Each child Repeating Group shall be modeled as a child table associated with its parent via the parent table identifier column.

8.7.3 Table Column Names

Table column names shall the same as Fact Type Business Friendly Names with spaces replaced with "_" and all capital letters. When the Friendly Name is all capital letters then an "_" will be generated between each capital character. For example, "Insured CEO Change" will become "INSURED_C_E_O_CHANGE". This is to be able to translate back to camel case on execution. Without the _ for caps the camel case conversion would be insuredCeoChange. By putting the underscore between two Capitals in the Friendly name the conversion to Camel Case will be insuredCEOChange.

An identifier column shall be defined for each Repeating Group table in the format “DECISION_ <Repeating Group Name>_id.

8.7.4 Example of a DV Repeating Group Model and the InfoHub Schema

For example, consider the following DV Repeating Group Structure:

Root

- Loan (RG)
    - Loan Type (FT)
    - Borrower (RG)
        - Borrower Age (FT)
        - Borrower Credit Rate (FT)

For this structure the InfoHub schema shall have the following tables:

DECISION_ROOT

- DECISION_ROOT_id

DECISION_LOAN

- DECISION_ROOT_id (FK to DECISION_ROOT)
- DECISION_LOAN_id
- LOAN_TYPE

DECISION_BORROWER

- DECISION_LOAN_id (FK to DECISION_LOAN)
- DECISION_BORROWER_id
- BORROWER_AGE
- BORROWER_CREDIT_RATE

8.8 Cache Management

DE can execute deployed artifacts extremely fast if they are loaded in advance into the Java heap memory. But it might take up a lot of memory, which will harm the general application performance.

From 7.1.2, DE prioritizes whether an executable artifact will be loaded into memory or evicted from it based on the last time it was executed and/or deployed. The artifacts caching is based on the configuration of the following properties in `application.properties`:

Name	Valid Values	Default value	Description
decision.de.mb.left.in.memory.artifacts.evict.trigger	Any positive number	300	Defines the MB left in memory threshold, which, when reached, will trigger an eviction process to evict unused or rarely used artifacts to make room for new artifacts.
decision.de.mb.left.in.memory.artifacts.evict.target	Any positive number higher than the trigger property above	400	Defines the eviction process target of MB left in memory to reach.

Name	Valid Values	Default value	Description
decision.de.unused.artifact.days.to.favor.new.deployment	Any positive number	7	Defines the minimum number of days an artifact is not executed to evict if reaching eviction threshold when trying to load new artifact deployment to memory. Set the value to 0 to disable.
decision.de.startup.cache.loading.latest.executed.days	Any positive number	7	Defines the number of days limit since last executed to include an executable in DE startup cache loading. Set the value to 0 to ignore and load on startup any executable, regardless of when last executed.

Name	Valid Values	Default value	Description
decision.de.startup.cache.loading.latest.deployed.days	Any positive number	7	Defines the number of days limit since last deployed to include an executable in the DE startup cache loading. Set the value to 0 to ignore and load on startup any executable, regardless of when last deployed.
decision.de.unused.artifact.days.to.keep.in.cache	Any positive number	30	Defines the number of days since the last deployment or execution of an executable to allow it to be kept in memory. This limit is enforced against all cached artifacts prior to every deployment. Set the value to 0 to disable.

Name	Valid Values	Default value	Description
decision.de.min.mb.left.in.memory.limit	Any positive number. Must be lower than the evict trigger property mentioned above.	100	Defines the minimum size of MB available in the Java heap memory at all times in order to avoid an out of memory event. Reaching this limit will reject any load to memory of an artifact, even for execution, if failing to evict and free memory above it.
decision.de.hours.since.executed.when.evicted.alarm	Any positive number	6	Defines the number of hours passed since an executable executed to the time it had to be evicted (to free memory) to issue an alarm.

Name	Valid Values	Default value	Description
decision.de.new.deployment.cache.strategy	EAGER, ONLY_ LATEST	EAGER	Defines the following load to cache strategy for new deployments: • EAGER: Cache all artifacts for every new deployment up to eviction threshold. • ONLY_ LATEST: Cache all artifacts of latest version assets only for every new deployment up to eviction threshold.

Name	Valid Values	Default value	Description
decision.de.publish.exec.audits.interval.millis	Any positive number	5000	Defines the milliseconds interval to collect and audit statistics of every execution sent to DE during this interval (See release notes for further description of the new execution statistics audit feature). In a single tenant environment this is audited in a designated log file 'execution-audit.log'. Set the value to 0 to disable executions auditing.

Name	Valid Values	Default value	Description
decision.de.publish.memory.events.interval.millis	Any positive number	5000	Defines the milliseconds interval to collect and audit memory events (cache load/evict events, memory thresholds reached, etc. See release notes for further description of the new memory events audit feature) occurred during this interval. In a single tenant environment this is audited in a designated log file "memory-event-audit.log". Set the value to 0 to disable memory events auditing.
decision.de.publish.exec.audits.one.batch.limit	Any positive number	5000	Defines the size limit of ended executions audit records that will be saved in one bulk.
decision.de.publish.memory.events.one.batch.limit	Any positive number	5000	Defines the size limit of memory events audit records that will be saved in one bulk.

Name	Valid Values	Default value	Description
decision.de.execution.info.log.auditing.level	INFO, DEBUG	DEBUG	Defines the log level used when logging each execution statistics audit in 'execution-audit.log'. Change the value to INFO for a single tenant environment.
decision.de.memory.event.log.auditing.level	INFO, DEBUG	DEBUG	Defines the log level used when logging each memory event audit in 'memory-event-audit.log'. Change the value to INFO for a single tenant environment.

8.9 Running Multiple and Single DE Instances

For high throughput and high availability, it is recommended to run multiple instances of DE behind a load balancer. However, when deploying a large number of instances, certain configurations are necessary to avoid overwhelming the database or exceeding the maximum number of database connections.

Cache Preloading at Startup

By default, at startup, each DE instance preloads the latest deployed package for every asset that was executed or deployed in the last 7 days into its in-memory cache. Starting a large number of DE instances simultaneously (for example, multiple DE pods in Kubernetes) can create a high load on the database.

To reduce this load, configure the system to preload packages for only the last 1 day:

- In `application.properties`, :

```
decision.de.startup.cache.loading.latest.deployed.days=1
decision.de.startup.cache.loading.latest.executed.days=1
```

To disable package preloading entirely, use:

```
decision.de.artifacts.load-on-startup.enabled=false
```

Note: Packages not preloaded will be cached on first execution, which may introduce a slight delay during initial access.

Managing Database Connection Pool Size

To prevent resource exhaustion when running multiple instances, reduce the database connection pool size:

- In `application.properties`:

```
spring.datasource.hikari.initial.pool.size=3        # default: 10
spring.datasource.hikari.minimum-idle=3             # default: 10
spring.datasource.hikari.maximum-pool-size=5        # default: 30
```

Configuration for a Single DE Instance

When running a single DE instance, you should disable the background polling process that checks for new deployments to load into the cache. This polling occurs every second by default and can unnecessarily increase CPU and database load.

To disable polling:

- In `application.properties`:

```
decision.de.db.repository.monitor.disabled=true
```

SAPIENS

Partnering for Success

Contact Us

For more information, please visit or contact us at:

info.sapiens@sapiens.com

For Support related queries, contact us at:

Decision.Support@sapiens.com

For Documentation related queries, contact us at:

Decision_documentation@sapiens.com

www.sapiensdecision.com

Partnering for Success